

PROJEKTANALYSE BASALTEMPERATUR

Stand: 04. Juni 2026, 15:40 Uhr Europe/Berlin

Projektpfad: /Users/kristian/Github/basaltemperatur

Kurzfazit

=====

Das Projekt ist technisch in einem weit fortgeschrittenen Zustand. Web-App, iOS-App, Supabase-Auth, Datenmodell, Premium-Entitlements, Stripe Checkout, StoreKit-2-IAP-Anbindung, rechtliche Seiten und App-Store-Grundlagen sind vorhanden. Die lokalen Build- und Testchecks sind erfolgreich.

Noch nicht final live-/app-store-reif sind vor allem operative Punkte: echte App-Store-Connect-Konfiguration, echtes StoreKit-Sandbox-End-to-End, korrekte App-Store-Screenshots, finale App-Privacy-Angaben, Testaccount, Review Notes, Distribution-Signing/Archive und einzelne produktive Keys.

Technischer Prüfstand

=====

Durchgeführte Checks

- npm run lint: erfolgreich.
- npm run test: erfolgreich, 5 Testdateien, 22 Tests.
- npm run build: erfolgreich.
- npm audit --omit=dev: 0 Vulnerabilities.
- iOS Simulator Tests: erfolgreich, 5 OvulationCalculatorTests.
- iOS Release Build ohne Code Signing: erfolgreich.
- Live-Checks:
 - https://basaltemperatur.online/ liefert HTTP 200.
 - https://www.basaltemperatur.online/ leitet auf die kanonische Domain um.
 - https://basaltemperatur.online/login liefert HTTP 200.
 - https://basaltemperatur.online/datenschutz liefert HTTP 200.
 - POST /api/checkout ohne Login liefert HTTP 401, erwartetes Verhalten.
 - POST /api/app-store/notifications ohne signedPayload liefert HTTP 400, erwartetes Verhalten.

Geänderte Punkte in dieser Prüfung

- Login-Redirect wird clientseitig auf interne Pfade begrenzt.
- Registrierungs-Erfolgsmeldung nach E-Mail-Bestätigung nutzt jetzt einen stabilen Message-Code.
- Loginseite zeigt die Check-Email-Meldung korrekt an.
- Upgrade-/Paywall-Button bleibt bei Checkout-Fehlern nicht mehr dauerhaft im Loading-Zustand und zeigt eine Fehlermeldung.

Wichtiger Hinweis zum Arbeitsbaum

Der Git-Arbeitsbaum war bereits vor dieser Analyse verändert. Ich habe keine fremden Änderungen zurückgesetzt. Auffällig sind u. a. bestehende Änderungen im iOS-Projekt, in mehreren Swift-Dateien, in Supabase-Temp-Dateien sowie ein untracked CLAUDE.md.

Funktioniert Login?

=====

Web

Der Login-Flow ist grundsätzlich funktionsfähig implementiert:

- Loginseite ist live erreichbar.
- Supabase Auth wird über @supabase/ssr und Client-Komponente verwendet.
- Geschützte Routen werden über proxy.ts abgesichert.
- Ungültige/nicht bestätigte Credentials werden abgefangen.
- Onboarding-Gating prüft onboarding_completed und sensitive_data_consent_at.
- OAuth Callback sanitizt den next-Parameter auf interne Pfade.

Nicht vollständig beweisbar ohne Testkonto:

- Erfolgreicher Login mit echtem Benutzer.
- E-Mail-Bestätigung mit produktiver Supabase-Redirect-Konfiguration.
- Passwort-Reset-E-Mail in produktiver Supabase-Mail-Konfiguration.
- Google OAuth, falls aktiv gewünscht.

iOS

Der iOS-Login-Flow ist grundsätzlich funktionsfähig implementiert:

- Login/Signup laufen gegen Supabase Auth REST.
- Tokens werden in der Keychain gespeichert.
- Session Refresh ist implementiert.
- Logout löscht lokale Tokens.
- Gesundheitsdaten-Einwilligung ist im Signup/Onboarding berücksichtigt.

Nicht vollständig beweisbar ohne Testkonto:

- Erfolgreicher Login auf echter Supabase-Instanz.
- E-Mail-Confirmation-Verhalten in iOS.
- Konto-Löschung mit echtem Nutzer.

Funktioniert die Paywall?

=====

Web-Paywall / Stripe

Der Web-Premium-Flow ist strukturell korrekt:

- /api/checkout ist geschützt und verlangt eingeloggte Nutzer.
- Checkout nutzt STRIPE_SECRET_KEY und STRIPE_PRICE_ID.
- Live-Stripe mit localhost-URL wird blockiert.
- Stripe Webhook prüft Signatur mit STRIPE_WEBHOOK_SECRET.
- checkout.session.completed setzt profiles.has_lifetime_access = true.
- Webhook prüft optional den erwarteten STRIPE_PRICE_ID.
- Premium-Entitlement-Felder werden per DB-Trigger gegen clientseitige Selbstfreischaltung geschützt.

Live verifiziert:

- /api/checkout ohne Login liefert 401.

Nicht vollständig beweisbar ohne Stripe-Testkauf:

- Echter Checkout-Session-Start mit eingeloggtem Nutzer.
- Webhook-Zustellung von Stripe in Produktion.
- Erfolgreiche Freischaltung nach echter Zahlung.
- Refund-/Chargeback-Folgeprozesse im Web.

iOS-Paywall / StoreKit 2

Der iOS-IAP-Flow ist strukturell korrekt:

- StoreKit-2-Produkt-ID: de.basaltemperatur.lifetime.
- Lokale StoreKit-Datei enthält NonConsumable "Basaltemperatur Vollzugang" für 99.
- PremiumStore lädt Product.products(for:).
- purchase() nutzt StoreKit-Verifikation und jwsRepresentation.
- restore() nutzt AppStore.sync() und Transaction.currentEntitlements.
- /api/app-store/entitlement verifiziert JWS serverseitig und setzt has_lifetime_access.
- /api/app-store/notifications ist erreichbar und erwartet signedPayload.
- App Store Server Notifications V2 Route ist vorhanden.

Nicht vollständig beweisbar ohne App Store Connect Sandbox:

- Das Produkt existiert in App Store Connect.
- Sandbox-Produkt ist nach Apple-Sync verfügbar.
- Kauf, Restore, Refund/Revoke funktionieren mit echten StoreKit-Sandbox-Transaktionen.

Premium-Gating

Gates sind vorhanden:

- Dashboard zeigt ohne Lifetime nur kostenlose Einträge/Kalender plus UpgradeBanner.
- Statistiken sind ohne Lifetime gesperrt.
- Zyklusvergleich redirectet ohne Lifetime.
- Export redirectet ohne Lifetime.
- Kalender zeigt Prognosen nur mit Lifetime und ausreichender Datenbasis.
- iOS zeigt PremiumPaywall in Dashboard, Kalender, Statistiken und PDF-Export.

Sicherheits-/Datenschutzprüfung
=====

Positiv

- RLS ist für temperature_entries und period_entries aktiviert.
- RLS-Policies begrenzen Datenzugriff auf auth.uid() = user_id.
- Entitlement-Felder werden durch prevent_client_entitlement_updates geschützt.
- Sensitive-Data-Consent wird per Trigger für Temperatur-/Periodenwrites verlangt.
- iOS speichert Tokens in Keychain.
- iOS Analytics ist Opt-in, nicht Default-on.
- PrivacyInfo.xcprivacy ist vorhanden und enthält UserDefaults Access Reason.
- App-Store-Release-Build blendet externen Web-Kauf-Link aus.
- NEXT_PUBLIC_SUPABASE_ANON_KEY ist öffentlich nutzbarer Anon-Key, kein Secret.
- SUPABASE_SERVICE_ROLE_KEY wird serverseitig verwendet und nicht in iOS benötigt.

Risiken / offene Punkte

- Echte Login-, Signup-, E-Mail-Confirm- und Passwort-Reset-Flows sind ohne Test account nicht end-to-end geprüft.
- Stripe und StoreKit sind ohne echte Sandbox-/Testtransaktionen nicht end-to-end bewiesen.
- .env.production enthält nach sichtbarer Struktur keine APP_STORE_BUNDLE_ID und

APP_STORE_LIFETIME_PRODUCT_ID; Fallbacks funktionieren, explizite Produktionseinträge sind trotzdem empfehlenswert.

- TRAFFIC_HASH_SALT ist in der Example-Datei nur Platzhalter; produktiv muss ein langer zufälliger Wert gesetzt sein.
- App-Store-Screenshots im Repo sind 640 x 640 und damit nicht App-Store-konform.
- iOS Target unterstützt aktuell iPhone und iPad (TARGETED_DEVICE_FAMILY = 1,2). Entweder iPad sauber unterstützen und Screenshots liefern oder bewusst iPhone-only konfigurieren.
- Code-Suche findet console.log/print-Debug-Ausgaben. Viele sind Service Worker, Scripts oder DEBUG-geschützt; vor Release trotzdem entscheiden, ob das gewünschte Logging ist.
- Es gibt persönliche/geschäftliche Fallback-Daten in legalConfig.ts und docs/LEGAL.md. Diese müssen vor Livegang final rechtlich geprüft werden.
- Kein echtes App-Archive mit App Store Distribution Signing wurde erzeugt; nur unsigned Release Build.

Code-Freiheit von Fehlern

=====

Der Code ist nicht "bewiesenermaßen fehlerfrei" - das kann keine statische Prüfung garantieren. Der aktuelle Zustand ist aber build- und testfähig:

- TypeScript/Next Build läuft durch.
- ESLint läuft durch.
- Unit Tests laufen durch.
- npm production audit meldet keine bekannten Vulnerabilities.
- iOS Unit Tests laufen durch.
- iOS Release Build läuft ohne Signing durch.

Bekannte verbleibende Testlücken:

- Kein automatisierter Browser-E2E-Test für Login/Registrierung/Checkout.
- Kein automatisierter Supabase-Integrationstest gegen eine Testdatenbank.
- Kein automatisierter Stripe-Webhook-Test gegen echte Stripe-Signaturen außer Unit-Mocks.
- Kein echter StoreKit-Sandbox-Test gegen App Store Connect.
- Kein TestFlight-Test auf physischem Gerät.
- Keine automatisierte Screenshot-/Layout-Prüfung für App Store Screenshots.

Was fehlt zum Onlinegehen?

=====

Web live

1. Produktive Environment-Variablen vollständig setzen:

- NEXT_PUBLIC_SUPABASE_URL
- NEXT_PUBLIC_SUPABASE_ANON_KEY
- SUPABASE_SERVICE_ROLE_KEY
- NEXT_PUBLIC_APP_URL=https://basaltemperatur.online oder https://www.basaltemperatur.online, konsistent mit Supabase Redirects
- STRIPE_SECRET_KEY
- STRIPE_PRICE_ID
- STRIPE_WEBHOOK_SECRET
- NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY, falls clientseitige Stripe-Flows genutzt werden
- TRAFFIC_HASH_SALT
- APP_STORE_BUNDLE_ID=de.basaltemperatur.app
- APP_STORE_LIFETIME_PRODUCT_ID=de.basaltemperatur.lifetime
- NEXT_PUBLIC_COMPANY_* final
- NEXT_PUBLIC_WEB_HOSTING_PROVIDER / LOCATION final

- NEXT_PUBLIC_DB_PROVIDER / LOCATION final
2. Supabase Dashboard prüfen:
 - Site URL auf öffentliche Domain setzen.
 - Redirect URLs für /auth/callback setzen.
 - E-Mail-Templates prüfen.
 - Google OAuth nur aktiv lassen, wenn Client ID/Secret korrekt gesetzt sind.
 - Alle Migrationen produktiv angewandt.
 - RLS Policies aktiv.
 3. Stripe Dashboard prüfen:
 - Live Price ID für Lifetime-Produkt.
 - Webhook Endpoint: <https://basaltemperatur.online/api/stripe-webhook>
 - Events mindestens checkout.session.completed.
 - Webhook Signing Secret in Produktion.
 - Testkauf mit Live/Test-Mode je Umgebung.
 4. Rechtliches prüfen:
 - Impressum, Datenschutz, AGB, Widerruf final.
 - Gesundheitsdaten-Einwilligung rechtlich final prüfen.
 - Keine medizinischen Claims wie Diagnose, Therapie, sichere Verhütung, garantierter Eisprung.

App Store / iOS

1. Apple Developer / App Store Connect:
 - Paid Apps Agreement, Tax und Banking abschließen.
 - App Record anlegen.
 - Bundle ID: de.basaltemperatur.app.
 - SKU vergeben, z. B. basaltemperatur-ios-001.
 - Kategorie wählen: wahrscheinlich Health & Fitness, ggf. Medical nur nach rechtlicher Prüfung vermeiden.
 - Altersfreigabe-Fragebogen ausfüllen.
2. In-App Purchase:
 - Non-Consumable anlegen.
 - Product ID: de.basaltemperatur.lifetime.
 - Reference Name: Basaltemperatur Vollzugang.
 - Display Name: Basaltemperatur Vollzugang.
 - Beschreibung max. 45 Zeichen, z. B. "Prognosen, Statistiken und Export".
 - Preisstufe für 9,99 EUR wählen.
 - App Review Screenshot für IAP hochladen.
 - Ersten IAP zusammen mit neuer App-Version einreichen.
3. StoreKit Sandbox:
 - Sandbox Tester anlegen.
 - Kauf testen.
 - Restore testen.
 - Refund/Revoke oder Notification-Test testen.
 - Backendprofil prüfen: has_lifetime_access, entitlement_source, app_store_original_transaction_id.
4. App Store Server Notifications:
 - Version 2 URL in App Store Connect setzen:
<https://www.basaltemperatur.online/api/app-store/notifications>
 - TLS/HTTPS ist erforderlich; Domain liefert HTTPS.
 - Sandbox- und Production-Notifications testen.
5. Screenshots:
 - Aktuelle Repo-Screenshots sind 640 x 640 und nicht ausreichend.
 - Apple verlangt 1 bis 10 Screenshots pro Plattform/Displaygröße in JPG/PNG.

- Da TARGETED_DEVICE_FAMILY = 1,2 ist, werden iPhone und iPad relevant.
- Entweder iPad-Screenshots liefern oder Target auf iPhone-only ändern.

6. Signing / Upload:

- App Store Distribution Signing einrichten.
- Archive in Xcode erstellen.
- Upload via Xcode Organizer oder Transporter.
- TestFlight Build verarbeiten lassen.
- TestFlight auf echtem Gerät prüfen.

App Store Connect Angaben

=====

Basisdaten

- App Name: Basaltemperatur
- Bundle ID: de.basaltemperatur.app
- SKU: noch festlegen
- Version: 1.0.0
- Build: 1
- Copyright: noch final angeben, z. B. 2026 Kristian Hoffmann
- Kategorie: Health & Fitness empfohlen, Medical nur mit Vorsicht
- Altersfreigabe: per Fragebogen, wahrscheinlich nicht Kids; gesundheitsbezogene Daten klar angeben
- Privacy Policy URL: <https://www.basaltemperatur.online/datenschutz>
- Support URL: <https://www.basaltemperatur.online/support>
- Marketing URL: optional, <https://basaltemperatur.online/>

App-Beschreibung Vorschlag

Basaltemperatur ist ein persönliches Zyklustagebuch für Temperaturwerte, Periodentage und Messqualität. Die App hilft, Temperaturverläufe übersichtlich zu dokumentieren und bei ausreichender Datenbasis statistische Hinweise zu Zyklusmustern, fruchtbaren Fenstern und Periodenbeginn anzuzeigen.

Wichtiger Hinweis: Die App ist kein Medizinprodukt, kein Verhütungsmittel und ersetzt keinen ärztlichen Rat.

Subtitle Vorschlag

Zyklus & Temperatur

Keywords Vorschlag

Basaltemperatur, Zyklus, Periode, Temperatur, Fertilität, Kalender, Eisprung

Promotional Text Vorschlag

Dokumentiere Temperatur, Periode und Messqualität. Prognosen erscheinen erst bei ausreichender Datenbasis.

Review Notes Vorschlag

Basaltemperatur ist ein persönliches Zyklustagebuch. Die App ist kein Medizinprodukt, kein Verhütungsmittel und kein Diagnosewerkzeug. Prognosen werden erst nach mindestens 3 abgeschlossenen Zyklen und ausreichenden

Temperaturwerten angezeigt.

Bitte verwenden:

- Testaccount: noch anlegen und hier eintragen.
- Passwort: noch eintragen.
- IAP: Non-Consumable de.basaltemperatur.lifetime.
- App Store Server Notifications V2:
<https://www.basaltemperatur.online/api/app-store/notifications>

Die Verarbeitung sensibler Gesundheitsdaten erfordert eine ausdrückliche Einwilligung. Diese kann in den Einstellungen widerrufen werden.

App Privacy Angaben

=====

Die App sammelt Daten. In App Store Connect daher "Yes, we collect data from this app" wählen.

Empfohlene Datenarten:

- Health: Basaltemperatur, Periodendaten, Zervixschleim, Messqualität.
Linked to user: Yes.
Tracking: No.
Purpose: App Functionality.
- Email Address:
Linked to user: Yes.
Tracking: No.
Purpose: App Functionality.
- Name:
Linked to user: Yes.
Tracking: No.
Purpose: App Functionality.
- User ID:
Linked to user: Yes.
Tracking: No.
Purpose: App Functionality und, bei Analytics Opt-in, Analytics.
- Product Interaction:
Linked to user: Yes.
Tracking: No.
Purpose: Analytics, wenn iOS-Opt-in bzw. Web-Analytics aktiv ist.
- Purchase History:
Linked to user: Yes.
Tracking: No.
Purpose: App Functionality.
- Coarse Location:
Nur angeben, wenn IP-/Traffic-Analytics serverseitig für Region/Standort verarbeitet oder gespeichert werden.
Linked to user: konservativ Yes, falls User-/Session-Bezug besteht.
Tracking: No.
Purpose: Analytics.

Privacy Manifest Stand:

- NSPrivacyTracking: false.
- NSPrivacyTrackingDomains: leer.
- UserDefaults Access Reason: CA92.1.

Benötigte Keys und Konfigurationen

=====

Unbedingt produktiv nötig

- NEXT_PUBLIC_SUPABASE_URL
- NEXT_PUBLIC_SUPABASE_ANON_KEY
- SUPABASE_SERVICE_ROLE_KEY
- NEXT_PUBLIC_APP_URL
- STRIPE_SECRET_KEY
- STRIPE_PRICE_ID
- STRIPE_WEBHOOK_SECRET
- TRAFFIC_HASH_SALT
- NEXT_PUBLIC_COMPANY_NAME
- NEXT_PUBLIC_COMPANY_STREET
- NEXT_PUBLIC_COMPANY_CITY
- NEXT_PUBLIC_COMPANY_COUNTRY
- NEXT_PUBLIC_COMPANY_EMAIL
- NEXT_PUBLIC_WEB_HOSTING_PROVIDER
- NEXT_PUBLIC_WEB_HOSTING_LOCATION
- NEXT_PUBLIC_DB_PROVIDER
- NEXT_PUBLIC_DB_LOCATION

Empfohlen explizit zu setzen

- APP_STORE_BUNDLE_ID=de.basaltemperatur.app
- APP_STORE_LIFETIME_PRODUCT_ID=de.basaltemperatur.lifetime
- NEXT_PUBLIC_APP_STORE_URL, sobald App-Store-Link existiert
- NEXT_PUBLIC_ENABLE_ANALYTICS=true/false bewusst festlegen

iOS Build Settings

- DEVELOPMENT_TEAM=W9V62XQNWZ ist gesetzt.
- PRODUCT_BUNDLE_IDENTIFIER=de.basaltemperatur.app ist gesetzt.
- MARKETING_VERSION=1.0.0.
- CURRENT_PROJECT_VERSION=1.
- TARGETED_DEVICE_FAMILY=1,2, also iPhone und iPad.
- WEB_APP_URL=https://www.basaltemperatur.online.
- ALLOW_WEB_UPGRADE_LINK=NO im Release-Kontext.
- SUPABASE_URL und SUPABASE_ANON_KEY sind im Build/Info.plist vorhanden.

Apple/App Store Keys

Nicht im aktuellen Code zwingend nötig:

- App Store Connect API Key (.p8) nur, wenn Uploads/Metadata per API oder Transporter-JWT automatisiert werden sollen.
- App Store Server API Issuer ID / Key ID / Private Key nur, wenn zusätzlich Apples Server API aktiv abgefragt wird. Die vorhandene Implementierung verifiziert JWS aus StoreKit/Notifications und benötigt dafür keinen privaten Apple-Key.

Benötigt in App Store Connect:

- Bundle ID.
- App Record.
- Non-Consumable Product ID.

- Sandbox Tester.
- App Store Server Notifications V2 URL.

Quellen

=====

Apple App Privacy Details:

<https://developer.apple.com/app-store/app-privacy-details/>

Apple Manage App Privacy:

<https://developer.apple.com/help/app-store-connect/manage-app-information/manage-app-privacy>

Apple In-App Purchase Overview:

<https://developer.apple.com/help/app-store-connect/configure-in-app-purchase-settings/overview-for-configuring-in-app-purchases/>

Apple Create Consumable or Non-Consumable IAP:

<https://developer.apple.com/help/app-store-connect/manage-in-app-purchases/create-consumable-or-non-consumable-in-app-purchases/>

Apple In-App Purchase Information:

<https://developer.apple.com/help/app-store-connect/reference/in-app-purchase-information>

Apple App Store Server Notifications V2:

<https://developer.apple.com/documentation/AppStoreServerNotifications/App-Store-Server-Notifications-V2>

Apple Upload Builds:

<https://developer.apple.com/help/app-store-connect/manage-builds/upload-builds/>

Apple Screenshot Specifications:

<https://developer.apple.com/help/app-store-connect/reference/screenshot-specifications>

Apple Set App Age Rating:

<https://developer.apple.com/help/app-store-connect/manage-app-information/set-an-app-age-rating>

Priorisierte To-do-Liste

=====

Blocker vor App Store Upload

1. App Store Connect App Record anlegen.
2. Non-Consumable IAP de.basaltemperatur.lifetime anlegen.
3. App Store Server Notifications V2 URL eintragen.
4. Sandbox Tester und Review-Testaccount anlegen.
5. Echte StoreKit Sandbox Tests durchführen.
6. App-Store-konforme Screenshots erstellen.
7. App Privacy Angaben ausfüllen und veröffentlichen.
8. Altersfreigabe-Fragebogen ausfüllen.
9. App Review Notes mit Testdaten ergänzen.
10. Distribution Signing/Archive/Upload durchführen.

Blocker vor Web-Live-Vertrieb

1. Stripe Live-Webhook final konfigurieren.
2. Stripe Testkauf gegen Produktiv-URL durchführen.

3. Supabase Auth Redirect URLs final prüfen.
4. Rechtliche Seiten final juristisch/fachlich prüfen.
5. TRAFFIC_HASH_SALT produktiv setzen.
6. APP_STORE_BUNDLE_ID und APP_STORE_LIFETIME_PRODUCT_ID in Produktion explizit setzen.

Empfohlen vor öffentlichem Launch

1. Playwright/E2E-Test für Login, Registrierung, Checkout-Fehlerpfad.
2. TestFlight-Test auf physischem iPhone.
3. iPad-Entscheidung treffen: iPhone-only oder iPad sauber bedienen.
4. Screenshots mit echten App-Store-Auflösungen erzeugen.
5. Debug-/ServiceWorker-Logging bewusst reduzieren oder akzeptieren.
6. Keine medizinischen Versprechen in Store-Texten und Screenshots.